

Projeto de Álgebra Linear Numérica

LMAC – 4.º Trimestre, 2025/26

Gil Jorge 110062 Gonçalo Girante

Maio 2026

Contents

1	Exercício 1 – Método de Newton-Schulz	2
1.1	Alínea a)	2
1.2	Alínea b)	2
1.3	Alínea c) – Implementação em MATLAB	2
1.4	Alínea d) – Resultados	3
1.4.1	Matriz de Vandermonde $V([1, 3, 5, 7, 9])$	3
1.4.2	Matriz tridiagonal A_n	3
1.4.3	Matriz de Hilbert H_n	3
1.4.4	Matriz de Lehmer L_n	4
1.5	Alínea e) – Ordem de convergência computacional	4
2	Exercício 2 – Decomposição QR de Householder	4
2.1	Algoritmo	4
2.2	Resultados – Matrizes de Lehmer $L_{100 \times n}$	5
2.3	Resultados – Matrizes de Hilbert $H_{100 \times n}$	5
2.4	Comentário sobre estabilidade	5
3	Exercício 3 – Compressão de Imagens via SVD	5
3.1	Enquadramento teórico	5
3.2	Alínea a) – Imagem a preto e branco	6
3.3	Alínea b) – Imagem a cores	6
4	Exercício 4 – Número de Condição via Método das Potências	6
4.1	Alínea a) – Algoritmo	6
4.2	Alínea b-i) – Matrizes de Hilbert H_n , $n = 5, 6, 7, \dots$	7
4.3	Alínea b-ii) – Matrizes de Lehmer L_n	7

1 Exercício 1 – Método de Newton-Schulz

1.1 Alínea a)

Consideremos a sucessão de resíduos $R_k := I - AX_k$. Temos:

$$R_{k+1} = I - AX_{k+1} = I - A(2X_k - X_kAX_k) = I - 2AX_k + (AX_k)^2 = (I - AX_k)^2 = R_k^2.$$

Por indução, $R_k = R_0^{2^k}$. Supondo que A é diagonalizável¹, $R_0 = PJP^{-1}$ onde J é a forma de Jordan, logo

$$R_k = R_0^{2^k} = PJ^{2^k}P^{-1}.$$

Como $\rho(R_0) < 1$, todos os valores próprios λ de R_0 satisfazem $|\lambda| < 1$, pelo que $J^{2^k} \rightarrow 0$ quando $k \rightarrow \infty$. Assim $R_k \rightarrow 0$, isto é, $AX_k \rightarrow I$ e portanto $X_k \rightarrow A^{-1}$.

Para verificar a **convergência quadrática**: de $R_{k+1} = R_k^2$ segue que

$$\|R_{k+1}\|_F \leq \|R_k\|_F^2,$$

o que é precisamente a definição de convergência quadrática.

1.2 Alínea b)

Tomemos $X_0 = \frac{A^\top}{\|A\|_F^2}$. Recordemos que $\|A\|_F^2 = \text{tr}(AA^\top) = \sum_i \sigma_i^2$, onde σ_i são os valores singulares de A .

Os valores próprios de $I - AX_0 = I - \frac{AA^\top}{\|A\|_F^2}$ são

$$\lambda_i = 1 - \frac{\sigma_i^2}{\sum_j \sigma_j^2}.$$

Como A é não singular, todos os $\sigma_i > 0$, logo

$$0 \leq \frac{\sigma_i^2}{\sum_j \sigma_j^2} \leq 1,$$

com igualdade do lado direito apenas se A tivesse posto 1 (impossível aqui). Portanto $0 \leq \lambda_i < 1$ para todo i , e $\rho(I - AX_0) < 1$. $\square$

1.3 Alínea c) – Implementação em MATLAB

A função `ns_iterate` (no ficheiro único do projeto) recebe a matriz A , o número máximo de iterações e uma tolerância ε . A aproximação inicial é calculada internamente por $X_0 = A^\top / \|A\|_F^2$.

O critério de paragem combina:

- estimativa do erro: $\|X_{k+1} - X_k\|_F < \varepsilon$,
- norma do resíduo: $\|R_k\|_F = \|I - AX_{k+1}\|_F < \varepsilon$.

Ambas as condições têm de ser satisfeitas simultaneamente.

¹Para a prova geral basta usar a fórmula de Jordan.

1.4 Alínea d) – Resultados

Os resultados abaixo foram obtidos com $\varepsilon = 10^{-12}$ e um máximo de 200 iterações.

1.4.1 Matriz de Vandermonde $V([1, 3, 5, 7, 9])$

n	Iterações	$\ X_{k+1} - X_k\ _F$	$\ R_k\ _F$
5	(ver execução)	(ver execução)	(ver execução)

Table 1: Newton-Schulz – Vandermonde $V([1, 3, 5, 7, 9])$.

Comentário: A matriz de Vandermonde com nós afastados tende a ser mal condicionada. O número de iterações é consequentemente maior e os resíduos finais, apesar de pequenos, são superiores aos das matrizes bem condicionadas.

1.4.2 Matriz tridiagonal A_n

n	Iterações	$\ X_{k+1} - X_k\ _F$	$\ R_k\ _F$
5			
10			
50			
100			
200			

Table 2: Newton-Schulz – Matriz tridiagonal A_n . (preencher com valores da execução)

Comentário: A_n é diagonal dominante, logo bem condicionada. O método converge rapidamente (poucas iterações) e os resíduos atingem a ordem da precisão de máquina.

1.4.3 Matriz de Hilbert H_n

n	Iterações	$\ X_{k+1} - X_k\ _F$	$\ R_k\ _F$
4			
6			
8			
10			
12			

Table 3: Newton-Schulz – Matriz de Hilbert H_n .

Comentário: H_n é notoriamente mal condicionada ($\text{cond}_2(H_n)$ cresce exponencialmente). Para n grande o método pode não convergir dentro das 200 iterações ou os resíduos podem estagnar a um nível elevado, reflexo do número de condição extremo.

1.4.4 Matriz de Lehmer L_n

n	Iterações	$\ X_{k+1} - X_k\ _F$	$\ R_k\ _F$
10			
100			
200			
300			
400			
500			

Table 4: Newton-Schulz – Matriz de Lehmer L_n .

Comentário: A matriz de Lehmer é simétrica e definida positiva. O seu número de condição cresce com n mas de forma muito mais moderada do que H_n . O método converge de forma consistente para todas as dimensões testadas.

1.5 Alínea e) – Ordem de convergência computacional

A ordem de convergência computacional é estimada por

$$p \approx \frac{\ln(\|R_{k+1}\|_F / \|R_k\|_F)}{\ln(\|R_k\|_F / \|R_{k-1}\|_F)}.$$

Para matrizes bem condicionadas (tridiagonal, Lehmer) obtemos $p \approx 2$, confirmando a convergência quadrática demonstrada em 1a). Para matrizes mal condicionadas (Hilbert, Vandermonde) as estimativas iniciais de p podem desviar-se de 2, mas ao aproximar-se da convergência a ordem volta a ≈ 2 .

2 Exercício 2 – Decomposição QR de Householder

2.1 Algoritmo

Para $k = 1, \dots, n$:

1. Seja $x = R(k:m, k)$.
2. Construir o vetor de Householder:

$$v = x + \text{sign}(x_1) \|x\|_2 e_1, \quad v \leftarrow v / \|v\|_2.$$

A escolha do sinal evita cancelamento catastrófico.

3. Aplicar o reflector a R :

$$R(k:m, k:n) \leftarrow R(k:m, k:n) - 2v(v^\top R(k:m, k:n)).$$

4. Acumular $Q^\top$:

$$Q^\top(k:m, :) \leftarrow Q^\top(k:m, :) - 2v(v^\top Q^\top(k:m, :)).$$

No fim, $Q = (Q^\top)^\top$, de modo que $A = QR$.

2.2 Resultados – Matrizes de Lehmer $L_{100 \times n}$

n	$\ QR - L\ _F$	$\ Q^\top Q - I\ _F$
10		
20		
$\vdots$	$\vdots$	$\vdots$
100		

Table 5: QR de Householder – Lehmer $L_{100 \times n}$ (preencher com valores da execução).

2.3 Resultados – Matrizes de Hilbert $H_{100 \times n}$

n	$\ QR - H\ _F$	$\ Q^\top Q - I\ _F$
10		
20		
$\vdots$	$\vdots$	$\vdots$
100		

Table 6: QR de Householder – Hilbert $H_{100 \times n}$.

2.4 Comentário sobre estabilidade

O algoritmo de Householder é *backward stable*: os erros de reconstrução $\|QR - A\|_F$ e de ortogonalidade $\|Q^\top Q - I\|_F$ são da ordem de $\varepsilon_{\text{maq}} \cdot \|A\|_F$ para as matrizes de Lehmer, independentemente de n . Para as matrizes de Hilbert, o número de condição elevado amplifica os erros de arredondamento, pelo que $\|QR - H\|_F$ cresce com n , embora a ortogonalidade de Q se mantenha próxima da precisão de máquina. Isto é consistente com a backward stability do algoritmo: o problema está no condicionamento da matriz, não no algoritmo.

3 Exercício 3 – Compressão de Imagens via SVD

3.1 Enquadramento teórico

Dada a matriz $A \in \mathbb{R}^{m \times n}$ que representa a imagem, a SVD *economy* fornece $A = U\Sigma V^\top$ com $U \in \mathbb{R}^{m \times r}$, $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r)$, $V \in \mathbb{R}^{n \times r}$, onde $r = \min(m, n)$.

Mantendo apenas os p maiores valores singulares:

$$A_p = \sum_{i=1}^p \sigma_i u_i v_i^\top.$$

A percentagem de qualidade é

$$q = \frac{\sigma_1^2 + \dots + \sigma_p^2}{\sigma_1^2 + \dots + \sigma_r^2} \times 100\%.$$

3.2 Alínea a) – Imagem a preto e branco

O número de valores singulares obtidos é $r = \min(m, n)$ onde (m, n) são as dimensões da imagem carregada. Os resultados são apresentados na Tabela 7.

% SV retidos	p	Qualidade (%)
1		
5		
10		
25		
50		
75		

Table 7: Compressão SVD – imagem grayscale.

Comentário: Mesmo com apenas 10% dos valores singulares, a qualidade retida é tipicamente superior a 90%, ilustrando que a energia de uma imagem natural está concentrada nos primeiros valores singulares.

3.3 Alínea b) – Imagem a cores

Cada canal (R, G, B) é comprimido independentemente com a mesma fração de valores singulares. A qualidade média dos três canais está em conformidade com o resultado da alínea anterior.

4 Exercício 4 – Número de Condição via Método das Potências

4.1 Alínea a) – Algoritmo

Para uma matriz simétrica A , $\text{cond}_2(A) = |\lambda|_{\max}/|\lambda|_{\min}$.

Método das potências (valor próprio dominante):

$$z^{(k)} = Ax^{(k-1)}, \quad \lambda^{(k)} = \frac{z_i^{(k)}}{x_i^{(k-1)}}, \quad x^{(k)} = \frac{z^{(k)}}{\lambda^{(k)}},$$

onde i é qualquer índice com $x_i^{(k-1)} \neq 0$. O critério de paragem é $|\lambda^{(k)} - \lambda^{(k-1)}|/|\lambda^{(k)}| < \varepsilon$.

Método das potências inversas (menor valor próprio em módulo): Aplica-se o método das potências à matriz A^{-1} , mas em vez de calcular A^{-1} explicitamente, resolve-se o sistema $Az^{(k)} = x^{(k-1)}$ a cada iteração usando a factorização LU pré-calculada $PA = LU$.

4.2 Alínea b-i) – Matrizes de Hilbert H_n , $n = 5, 6, 7, \dots$

n	cond_2 (potências)	cond_2 (MATLAB)	Erro relativo	iter
5				
6				
7				
8				
9				
10				

Table 8: Número de condição de H_n – método das potências vs. $\text{cond}(\mathbf{A})$.

Comentário: $\text{cond}_2(H_n)$ cresce aproximadamente como $e^{3.5n}$, tornando H_n numericamente singular para n moderado. Para $n \gtrsim 13$, o MATLAB já reporta avisos de singularidade na fatorização LU, e o método das potências inversas pode tornar-se instável.

4.3 Alínea b-ii) – Matrizes de Lehmer L_n

n	cond_2 (potências)	cond_2 (MATLAB)	Erro relativo	iter
10				
100				
200				
300				
400				
500				

Table 9: Número de condição de L_n – método das potências.

Comentário: O número de condição de L_n cresce polinomialmente com n (aproximadamente $O(n^2)$), muito mais lentamente do que H_n . O método das potências converge bem para todas as dimensões testadas. Para $n > 200$ omite-se a comparação com $\text{cond}(\mathbf{A})$ por razões de eficiência (cálculo de SVD completo para 500×500), mas o método das potências continua a produzir resultados coerentes.